As you can imagine, analyzing this function is not straightforward. But, and here is the fun part, now you can certainly plot the function using your (possibly newly acquired) Jupyter Notebook skills; turns out that, indeed, plotting this function of `D` for a few values of `n`, we get the charts in figure 2.25, showing that, despite the single calls to `_push_down` will be slower with a higher branching factor, the total number of swaps is expected to decrease.

So, mystery solved, `_heapify` does behave as expected – and good news too, it gets faster as we increase the running time.

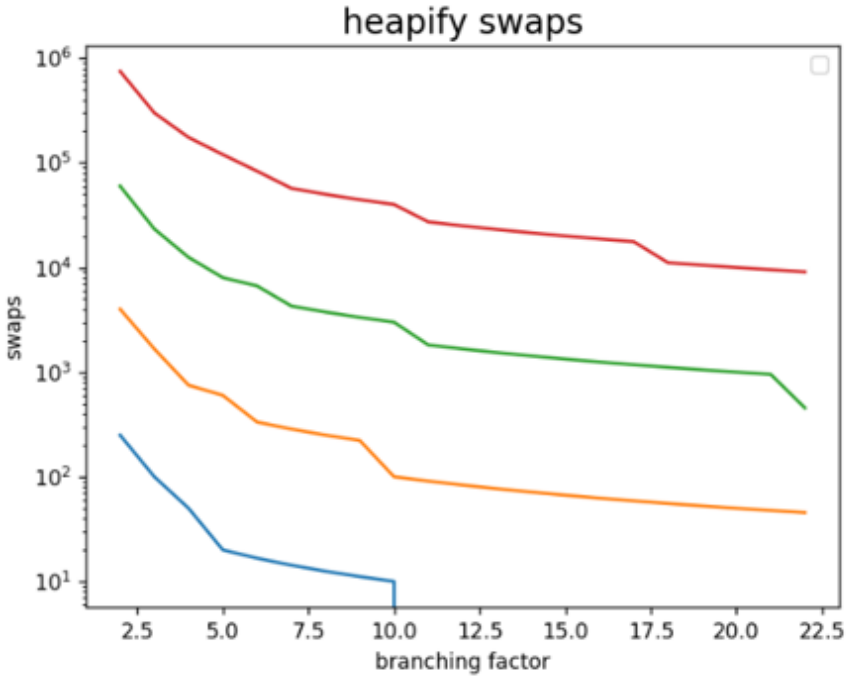


Figure 2.25 The upper bound for the number of swaps needed by method `heapify`, as a function of the branching factor `D`. Each line is a different value of `n`. Notice that the `y` axis uses a logarithmic scale.